

```
-- JA estavamos usando isso o tempo, o proprio
string eh um Type String = [Char]
type Produto =(Integer,String,Double)
type Cliente = (Integer, String, Double)

type Assoc k v = [(k, v)]
find :: Eq k => k -> Assoc k v -> v
find k t = head [v | (k', v) <- t, k' == k] -- vou
procurano meu array associativa - toda dyupla que
estiver igual a k quero ele

preco :: Produto -> Double
preco (_, _, p) = p

pago:: Cliente -> Double
pago (_, _, p) = p

atualizaPreco :: Produto -> Double -> Produto
atualizaPreco (idProd, nome,preco) inflacao =
(idProd, nome, preco * (1 + inflacao))

troco :: Produto -> Cliente-> Double
troco p c = abs( pago c - preco p )

-- Tipo somma

-- Bool, True e False já existem no Prelude. Não
precisamos redefini-los.

data Dir = Norte | Sul | Leste | Oeste deriving
Show

--deriving: da a possibilidae de imprimir esse tipo

type Coord = (Int, Int)
type Passo = Coord -> Coord
```

```
para :: Dir -> Passo
para Norte (x,y) = (x, y + 1)
para Sul (x,y) = (x, y - 1)
para Leste (x,y) = (x + 1, y)
para Oeste (x,y) = (x - 1, y)

caminhar :: [Dir] -> Passo
caminhar ds coord = foldl (flip para) coord ds
-- foldl nada mais eh do que isso aqui
-- caminhar [] coord = coord
-- caminhar (d:ds) coord = caminhar ds (para d
coord)

-- Tipo Produto

data Ponto = MkPonto Double Double deriving Show
-- Cada valo do tipo ponto vi conte dois valores
do tipo double

dist :: Ponto -> Ponto -> Double
dist (MkPonto x y ) (MkPonto x' y') = sqrt $ (x-
x')^2 + (y-y')^2

data Forma = Circulo Ponto Double | Retangulo Ponto
Double Double deriving Show

quadrado :: Ponto -> Double -> Forma
quadrado p l = Retangulo p l l

-- Tipos parametrizados

data Identidade a = Id a
-- o tipo ponto precisa definir um ponto a e ele
vai guardar 2 valores desse tipo depende do que
eh esse tipo

-- Maybe, Just e Nothing já existem no Prelude.
```

```
maybeDiv :: Int -> Int -> Maybe Int
maybeDiv _ 0 = Nothing
maybeDiv x y = Just (x `div` y) -- se y for 0 ele
nao vai retornar nada, se nao ele vai retornar o
valor da divisao

maybeHead :: [a] -> Maybe a
maybeHead [] = Nothing
maybeHead (x:xs) = Just x

divComErro :: Int -> Int -> Int
divComErro m n =
  case maybeDiv m n of
    Nothing -> error "Divisao por zero"
    Just x -> x

-- data Either a b = Left a | Right b deriving Show
(tipoa aparametrico acita dois tirpo e aceita dois
tipos e pode ser um ou outro )

eitherDiv :: Int -> Int -> Either String Int
eitherDiv _ 0 = Left "Divisao por zero"
eitherDiv x y = Right (x `div` y)
-- Fuzzy = verdadeiro falso pertinencia double
--fuzzy fica == False se o vamor menor ou igauila
ser veredri maior ou iguakl a zero

data Fuzzy = Verdadeiro | Falso | Pertinencia
Double deriving Show

fuzzyfica :: Double -> Fuzzy
fuzzyfica x
  | x <= 0 = Falso
  | x >= 1 = Verdadeiro
  | otherwise = Pertinencia x

f :: Identidade a -> a
f (Id x) = x
-- Tipo Recursivo
```

```
--Tipos que sao fefivndos udando o mprorpro tipo
que estamos definindo

data Lista a = Vazia | Cons a (Lista a) deriving
Show
-- cons 1 (Cons 2 (Cons 3 Vazia)) == [1,2,3]

-- Uma arvore binaria pode ser uma folha com um
valor (Leaf)
-- ou um no (Node) com um valor e duas subarvores:
esquerda e direita.
-- 0 tipo e recursivo porque cada subarvore tambem
e uma Tree a.
data Tree a = Leaf a | Node {_left :: Tree a,
                             _value :: a,
                             _right :: Tree a
                           } deriving Show

t :: Tree Int
t = Node t1 5 t2
  where
    t1 = Node (Leaf 1) 3 (Leaf 4)
    t2 = Node (Leaf 6) 7 (Leaf 8)
{-
      5
     / \
    3   7
   /\  /\
  1 4 6 8
-}

contem :: Ord a => Tree a -> a -> Bool
contem (Leaf y) x = x == y
contem (Node l y r) x | x == y = True
                     | x < y = contem l x
                     | x > y = contem r x

-- Record Type
data Ponto3D = Ponto { coordX :: Double, coordY ::
Double, coordZ :: Double } deriving Show
```

